

OHM'S CLUB

OHMIE BUILD GUIDE

Two games for an Arduino Uno R3, a 2.42" OLED, a slide potentiometer and a buzzer — wired once, flashed from a single repo. Everything here has been compiled and checked against the real toolchain.



RESIST · fight the current
FLAPPY OHM · fly the resistors

github.com/jinglebells2020/flappy-ohmie
Arduino Uno R3 · ATmega328P

— What's in here

- | | | | |
|---|--------------------------|----|---------------------------|
| 1 | Parts you need | 7 | Compile and flash |
| 2 | Wiring | 8 | RESIST — how it plays |
| 3 | Before you power up | 9 | Flappy Ohm — how it plays |
| 4 | Install the toolchain | 10 | When it doesn't work |
| 5 | Clone the repo | 11 | Making it yours |
| 6 | Check the pot and buzzer | 12 | Full pin map |

The two games

RESIST — Ohmie is a resistor, so he does a resistor's job. Current streams down a wire at him and your fader is his resistance. Resist too little and the current sweeps him off the wire; resist too much and $P = I^2R$ cooks him. You ride the gap between the two.

Flappy Ohm — the fader is Ohmie's height, and you fly him through a field of resistors. No tapping; you slide.

Both run on identical wiring, so you build the hardware once.

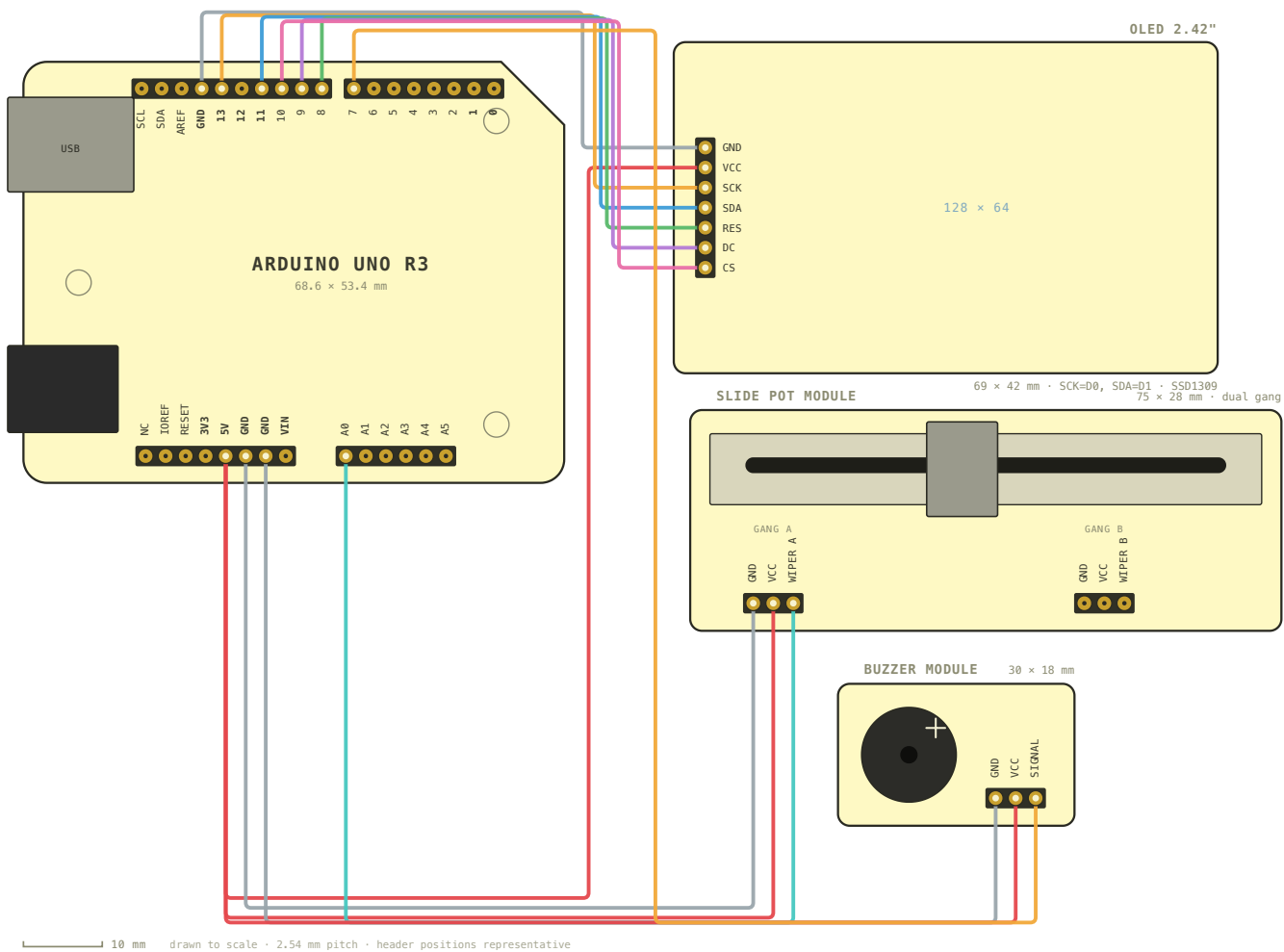
1 Parts you need

PART	WHAT TO LOOK FOR
Arduino Uno R3	Genuine or clone. Clones often report an unusual USB vendor ID, which only means you pass the board type explicitly — covered in section 7.
2.42" OLED, 128×64	SSD1309 controller, 7 pins (GND, VCC, SCK, SDA, RES, DC, CS). SSD1306 panels also work with a one-line change.
Slide pot module	Dual gang, 6 pins as two 3-pin headers: GND, VCC, WIPER per gang. A bare 3-pin fader works too.
Buzzer module	3 pins: GND, VCC, SIGNAL. A bare 2-pin buzzer works too — see the note in section 2.
Jumper wires	13 male-to-female, or male-to-male plus a breadboard.
USB cable	Type-B for a classic Uno.

PASSIVE OR ACTIVE BUZZER?

The sketches use `tone()`, which sweeps pitch only on a **passive** buzzer. Many 3-pin modules carry an **active** one, which has its own oscillator and will sound at a single fixed pitch no matter what the sketch asks for. Both are safe to wire exactly the same way, and section 6 tells you which you have.

2 Wiring



Boards drawn to true relative scale · 2.54 mm header pitch · 4.06 mm across the Uno's digital headers

OLED → Uno

OLED	UNO
GND	GND
VCC	5V
SCK / D0	D13 <i>fixed</i>
SDA / D1	D11 <i>fixed</i>
RES	D8
DC	D9
CS	D10

Slide pot → Uno

POT (GANG A)	UNO
GND	GND
VCC	5V
WIPER A	A0

Buzzer → Uno

BUZZER	UNO
GND	GND
VCC	5V
SIGNAL	D7

Leave gang B of the pot unconnected — one gang is all the game reads. If you have a bare 2-pin buzzer instead of a module, it's simply `+ → D7` and `- → GND` .

D13 AND D11 CANNOT BE MOVED

They are wired to the ATmega328P's SPI hardware in silicon. Put the clock on any other pin and the screen simply stays black. If D13 is genuinely unavailable, switch to I²C (`OLED_BUS 0` , `SCL→A5`, `SDA→A4`) rather than software SPI — software SPI is bit-banged and far too slow to look good.

D12 LOOKS FREE BUT ISN'T

D12 is MISO. While SPI is enabled in master mode the chip forces that pin to be an input, so anything you drive there is silently swallowed. The sketches refuse to compile if you put the buzzer on D11, D12 or D13.

3 Before you power up

Check what your OLED module says about **VCC**, on the silkscreen or in the listing you bought it from.

- **Marked 3.3–5 V** — the common case. Wire VCC to the Uno's 5 V pin and carry on.
- **Marked 3.3 V only** — run VCC from the Uno's 3.3 V pin and put a 1 k Ω resistor in series with each of the five signal lines (SCK, SDA, RES, DC, CS). Uno logic is 5 V and will otherwise stress a 3.3 V-only panel.

Then check the rest before applying power: nothing bridging 5 V to GND, the pot's two outer pins on 5 V and GND with the wiper on A0, and no wire in D0 or D1 — those are the USB serial pins and anything on them breaks uploading.

4 Install the toolchain

You need two things: a compiler for AVR boards, and the **U8g2** display library. Pick either route.

Route A — Arduino CLI (what this guide uses)

```
# macOS
brew install arduino-cli

# Linux
curl -fsSL https://raw.githubusercontent.com/arduino/arduino-cli/master/install.sh | sh

# Windows: download the .zip from arduino.github.io/arduino-cli
```

Then, on any platform:

```
arduino-cli config init
arduino-cli core update-index
arduino-cli core install arduino:avr
arduino-cli lib install "U8g2"
```

Route B — Arduino IDE

1. Install the Arduino IDE from arduino.cc/en/software.
2. **Tools → Board → Arduino AVR Boards → Arduino Uno.**
3. **Tools → Manage Libraries**, search **U8g2** by oliver, install.
4. Open the `.ino` you want and press Upload.

5 Clone the repo

```
git clone https://github.com/jinglebells2020/flappy-ohmie.git
cd flappy-ohmie
```

No Git installed? Open github.com/jinglebells2020/flappy-ohmie in a browser, press **Code** → **Download ZIP**, and unzip it.

What's inside

PATH	WHAT IT IS
Resist/	The current-fighting game. Start here.
Flappy0hm/	Fly Ohmie through resistors.
PinCheck/	Confirms the pot and buzzer before you load a game.
wiring-bench.html	Interactive wiring builder. Open it in any browser, drag boards, run cables, export a transparent PNG.
wiring-diagram.svg	The schematic on the previous page.

Each sketch lives in a folder matching its filename, which is what the Arduino toolchain expects. Point the compiler at the *folder*, not the `.ino`.

6 Check the pot and buzzer

Do this before loading a game. It needs only the pot and buzzer wired — no OLED — and it tells you in seconds whether your slider sweeps properly and which kind of buzzer you have.

Find your board's port:

```
arduino-cli board list
```

You're looking for something like `/dev/cu.usbmodem101` on macOS, `/dev/ttyACM0` on Linux, or `COM3` on Windows. Use it below:

```
arduino-cli compile --fqbn arduino:avr:uno --upload -p YOUR_PORT PinCheck
arduino-cli monitor -p YOUR_PORT -c baudrate=9600
```

WHAT YOU SEE	WHAT IT MEANS
Bar sweeps the full width, buzzer pitch rises as you slide	Both parts are good. Move on to section 7.
Number never moves	A0 is on an end pin, not the wiper. Swap A0 with one of the other two.
Number jumps 0 ↔ 1023 with nothing between	5 V and GND are on the wrong pins, or you picked pins from two different gangs.
Stops short, e.g. 40–980	Normal for a cheap fader. Widen <code>POT_LO</code> / <code>POT_HI</code> in the game sketch to match.
Buzzer beeps but the pitch never changes	You have an active buzzer. Harmless — the games still sound, just at one pitch.
Silent	Check polarity, and that SIGNAL is on D7 and VCC has 5 V.

CLOSE THE MONITOR BEFORE UPLOADING

A serial monitor holds the port, and the uploader cannot reset the board through a port something else owns. This is the single most common cause of `programmer is not responding`.

7 Compile and flash

Wire the OLED now as well, then:

```
arduino-cli compile --fqbn arduino:avr:uno Resist
```

You should see roughly:

```
Sketch uses 21094 bytes (65%) of program storage space.  
Global variables use 1555 bytes (75%) of dynamic memory,  
leaving 493 bytes for local variables.  
Low memory available, stability problems may occur.
```

THAT WARNING IS EXPECTED

1 KB of the RAM is the display framebuffer, and U8g2 links in another 96 bytes of I²C buffers even on SPI. Peak stack for these sketches is around 200 bytes, so there is roughly 2× headroom. Nothing is wrong.

Now flash it:

```
arduino-cli compile --fqbn arduino:avr:uno --upload -p YOUR_PORT Resist
```

Swap `Resist` for `Flappy0hm` to load the other game. Always pass `--fqbn arduino:avr:uno` explicitly — many Uno clones report a USB vendor ID the tooling doesn't recognise, so auto-detection reports the board as *Unknown* even though it is a perfectly ordinary ATmega328P.

Uploading from the Arduino IDE instead? Open `Resist/Resist.ino`, pick **Arduino Uno** and your port under **Tools**, and press Upload.

8 RESIST — how it plays

Current **I** streams at Ohmie from the right. Your slider sets his resistance **R**.

- **$I > R$** — the current wins and shoves him downstream. Off the left edge and you are **SWEPT AWAY**.
- **$P = I^2R$** — resisting is work, and work is heat. He glows from the inside; fill the heat bar and you **BURN OUT**.

The notch on the **R** bar marks where **$R = I$** , the point where he holds his ground. During a surge that notch is also exactly where he cooks fastest, so you have to give ground, cool off, and take it back in the lull. That push-and-recover rhythm is the whole game.

Surges get stronger and arrive sooner, and the ramp never plateaus — so every run ends, and skill decides when. Simulated against the real physics:

HOW YOU PLAY	OUTCOME
Never really move the fader	swept at ~11 s
Slam it to maximum and leave it	burned out at ~35 s
Match R to I, ignore the heat	burned out at ~40 s
Match I, back off when hot	43–55 s

Slide the fader to start, and again to retry after a crash. Your best score is kept in EEPROM and survives a power cycle.

9 Flappy Ohm — how it plays

Your slider position is Ohmie's height, with a little spring so he glides rather than snaps. Thread the resistors. Every one cleared is a point, a rising blip and a ring pop.

Difficulty ramps: the world speeds up and the gap tightens from 30 px to 21 px. Collision uses a 9×9 box at his centre, so his feet and antenna pass through harmlessly — deliberately forgiving, because near-misses film better than deaths.

Every gap the generator can produce was checked for a reachable centre. The tightest window in the whole difficulty curve is 10 px, and the spring closes the worst 22 px jump to within 0.04 px in the time available at top speed. It is always physically possible — reaction time is the only limit.

10 When it doesn't work

SYMPTOM	FIX
<code>programmer is not responding</code> <code>not in sync: resp=0x00</code>	A serial monitor is holding the port. Close it and retry. If it persists, press reset on the Uno just as the upload starts.
Screen totally black	Clock must be on D13 under hardware SPI. Then try <code>OLED_SSD1309 0</code> — wrong controller is the most common cause. Then check RES/DC/CS aren't swapped.
Ohmie moves the wrong way	<code>SLIDER_FLIP 1</code> , or swap the pot's 5 V and GND ends.
Can't reach the very top or bottom	Widen <code>POT_LO</code> / <code>POT_HI</code> — try 0 and 1023.
Ohmie jitters when you hold still	Noisy fader. Add a 100 nF capacitor from A0 to GND.
Motion looks choppy	Set <code>SHOW_FPS 1</code> ; the title screen prints render time. Over 20 ms means the bus is the bottleneck — get onto <code>OLED_BUS 1</code> .
Garbage or tearing	You're on I ² C with a long cable. Shorten it or move to SPI.
No sound at all	Check SIGNAL is on D7 and the module has 5 V and GND. A 2-pin buzzer needs correct polarity.
Board type not detected	Normal on clones. Always pass <code>--fqbn arduino:avr:uno</code> and <code>-p YOUR_PORT</code> explicitly.
Sketch won't compile, complains about <code>PIN_BUZZ</code>	Working as intended — you put the buzzer on D11, D12 or D13. Use D2–D7.

11 Making it yours

Every setting worth touching sits at the top of each sketch.

SETTING	WHAT IT DOES
OLED_BUS	<code>1</code> hardware SPI (~50 fps, the default), <code>0</code> I ² C (~30 fps), <code>2</code> software SPI (~10 fps).
OLED_SSD1309	<code>1</code> for a 2.42" SSD1309 panel, <code>0</code> for SSD1306.
SLIDER_FLIP	Reverses the fader direction.
SHOW_FPS	Prints render time on the title screen.
PIN_BUZZ	Buzzer pin. D2–D7 are free; D7 keeps the wiring at one end of the header.
OHM_R	Ohmie's size. Everything else scales from it — he is drawn from primitives, not a bitmap.

RESIST difficulty

`DRIFT` is how hard the current shoves, `HEAT_GAIN` and `HEAT_COOL` set how fast he cooks and recovers. Sustainable power is $\text{HEAT_COOL} / \text{HEAT_GAIN}$ — raise it and the game gets forgiving, lower it and the squeeze arrives sooner.

Flappy Ohm difficulty

`GAP_START` and `GAP_MIN` are the gap heights, and the `speed` line sets the ramp. If you change `OHM_R`, `OHM_HIT` or `GAP_MIN`, re-check that every gap is still reachable rather than trusting your eye — a hitbox that no longer fits produces gaps that cannot be passed at all.

12 Full pin map

UNO PIN	GOES TO	NOTES
5V	OLED VCC, pot VCC, buzzer VCC	Three wires share this pin.
GND	OLED GND, pot GND, buzzer GND	The Uno has three GND pins; spread the load.
A0	Pot WIPER A	The only analog read in either game.
D7	Buzzer SIGNAL	Any of D2–D7 works.
D8	OLED RES	Ordinary GPIO, reassignable.
D9	OLED DC	Ordinary GPIO, reassignable.
D10	OLED CS	Ordinary GPIO, reassignable.
D11	OLED SDA / MOSI	Fixed by SPI hardware.
D12	—	MISO. Forced to an input by SPI; unusable as an output.
D13	OLED SCK / D0	Fixed by SPI hardware.
D0 / D1	—	USB serial. Leave empty or uploads break.

Build sizes

Resist — 21094 B flash (65%), 493 B RAM free

FlappyOhm — 19984 B flash (61%), 523 B RAM free

PinCheck — 4046 B flash (12%), 1839 B RAM free

Target — 32256 B flash, 2048 B RAM

Ohmie is the Ω from *Ohm's* turned into a creature: a filled disc, hollowed out, the bottom cut away, the omega's own feet kept as his feet, and the apostrophe of *Ohm's* kept as an antenna. Nothing is a bitmap — change `OHM_R` and he resizes.

github.com/jinglebells2020/flappy-ohmie